PRAKTIKUM 1
Introduction and Fundamental Concepts

NAMA	: Ihsanta Zaki Sanjaya
KELAS	: 2 D4 Teknik Komputer B
NRP	: 3223600058
DOSEN	: Prof. Ir.Dadet Pramadihanto,M.Eng,Ph.D
MATA KULIAH	: Praktikum Pengolahan Sinyal dan Citra
TANGGAL	: Kamis, 20 Februari 2025

D4 TEKNIK KOMPUTER
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA
2025

Digital Signal Processing

I. TEORI DASAR

Sinyal dapat diartikan sebagai fungsi yang memuat informasi. Pemrosesan sinyal merupakan tindakan untuk mengolah sebuah sinyal secara matematis dengan berbagai tujuan dan penerapan. Salah satunya adalah *Digital Signal Processing* (DSP), yang berfokus pada teknik berbasis komputer dalam menganalisis, memodifikasi, dan menggambarkan sinyal dengan mencakup berbagai operasi matematis, seperti transformasi sinyal, konvolusi, dan korelasi. Sinyal dapat dikelompokkan menjadi sinyal waktu kontinu dan sinyal waktu diskrit. Sinyal yang mempunyai nilai di setiap titik dalam domain waktu disebut dengan sinyal kontinu. Sedangkan sinyal yang hanya bisa dijumpai pada titik-titik waktu tertentu disebut dengan sinyal diskrit. Agar sinyal kontinu dapat diproses secara digital, proses sampling diperlukan dengan cara mentransformasikan sinyal kontinu menjadi sinyal diskrit.

Dalam representasi sinyal dasar, terdapat beberapa jenis sinyal yang sering digunakan dalam DSP. Seperti sinyal unit impulse, unit step, serta sinyal sinus dan eksponensial kompleks. Beberapa sinyal ini dijadikan sebagai dasar dalam transformasi dan operasi matematika dalam DSP.

Sistem *Linear Time-Invariant* (LTI) merupakan model sistem yang dimanfaatkan dalam pemrosesan sinyal serta memiliki sifat kelinieran dan *time-invarience*. Kelinieran menunjukkan bahwa prinsip superposisi dan homogenitas dapat diterapkan dalam sistem, sedangkan *time-invarience* menunjukkan bahwa output tidak akan berubah meskipun terdapat pergeseran waktu pada inputnya. Untuk menganalisis sistem ini, dapat digunakan operasi konvolusi untuk memahami bagaimana suatu sinyal diproses oleh sistem. Korelasi juga dapat digunakan untuk menakar tingkat kemiripan di antara kedua sinyal.

Teorema sampling merupakan prinsip yang penting dalam DSP yang menguraikan bahwa sinyal analog dapat dikonversi menjadi sinyal digital dan sebaliknya tanpa kehilangan esensial di dalamnya. Dalam teorema ini, frekuensi sampel harus setidaknya dua kali lipat lebih tinggi dari frekuensi maksimum dalam sinyal asli agar informasi tetap terjaga. Jika frekuensi pengambilan sampel lebih rendah dari batas tersebut, maka akan timbul distorsi atau yang umum disebut sebagai aliasing. Untuk menghindarinya, anti-aliasing filter digunakan sebelum proses sampling dilakukan.

II. PERCOBAAN

1. Visualisasi sinyal

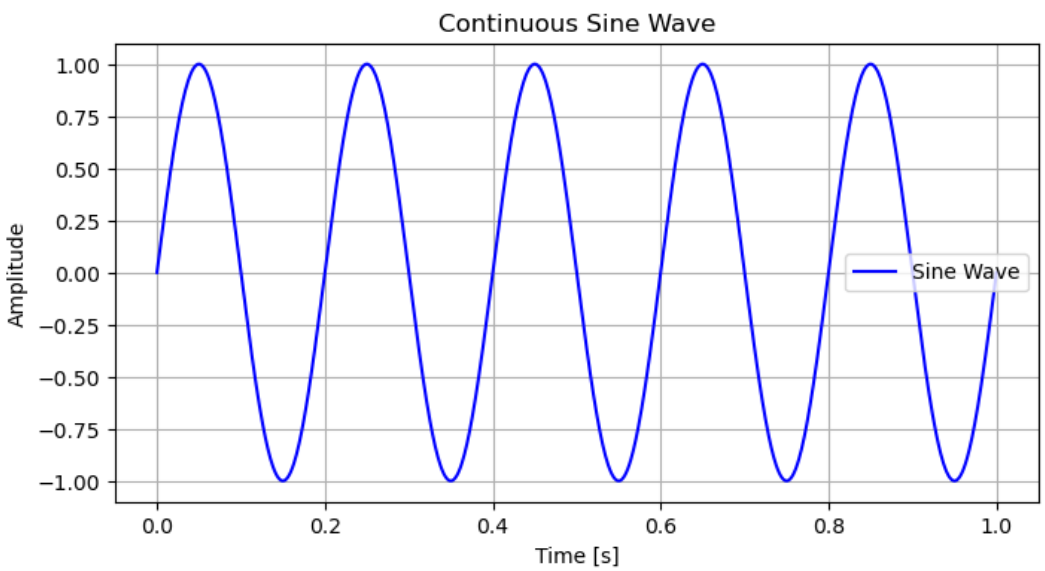
a. Plotting a Continuous Sine Wave

- f = 5 hz

```
import numpy as np
import matplotlib
matplotlib.use('TkAgg')
import matplotlib.pyplot as plt

# Define parameters for the sine wave
f = 5  # frequency in Hz
t = np.linspace(0, 1, 1000)  # time vector from 0 to 1 second
x_continuous = np.sin(2 * np.pi * f * t)

# Plot the continuous sine wave
plt.figure()
plt.plot(t, x_continuous)
plt.title('Continuous Sine Wave')
plt.xlabel('Time [s]')
plt.ylabel('Amplitude')
plt.grid(True)
plt.show()
```

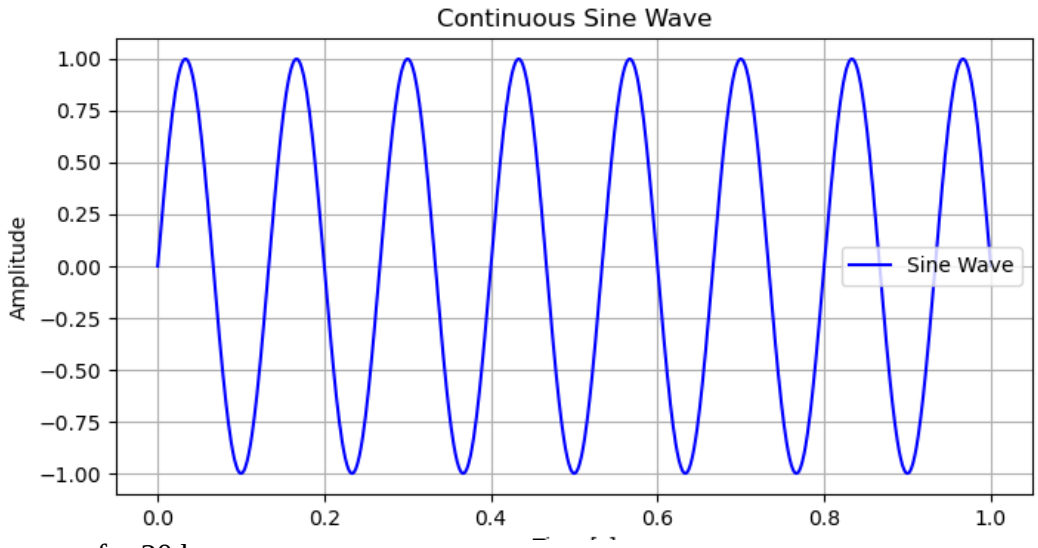


- f = 7.5 hz

```
import numpy as np
import matplotlib.pyplot as plt

# Define parameters for the sine wave
f = 7.5  # Frequency in Hz
t = np.linspace(0, 1, 1000)  # Time vector from 0 to 1 second
x_continuous = np.sin(2 * np.pi * f * t)

# Plot the continuous sine wave
plt.figure(figsize=(8, 4))  # Pastikan ukuran figure jelas
plt.plot(t, x_continuous, label="Sine Wave", color="b")
plt.title("Continuous Sine Wave")
plt.xlabel("Time [s]")
plt.ylabel("Amplitude")
plt.legend()
plt.grid(True)
plt.show()
```

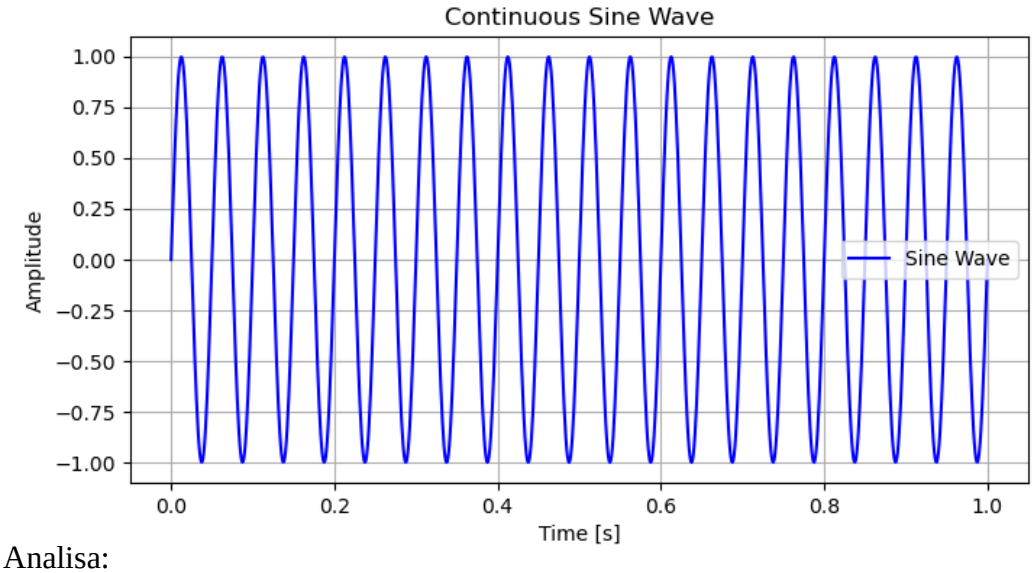


- f = 20 hz

```
import numpy as np
import matplotlib.pyplot as plt

# Define parameters for the sine wave
f = 20  # Frequency in Hz
t = np.linspace(0, 1, 1000)  # Time vector from 0 to 1 second
x_continuous = np.sin(2 * np.pi * f * t)

# Plot the continuous sine wave
plt.figure(figsize=(8, 4))  # Pastikan ukuran figure jelas
plt.plot(t, x_continuous, label="Sine Wave", color="b")
plt.title("Continuous Sine Wave")
plt.xlabel("Time [s]")
plt.ylabel("Amplitude")
plt.legend()
plt.grid(True)
plt.show()
```



Analisa:

Di dalam percobaan ini, dilakukan visualisasi gelobang sinus kontinu yang memiliki bentuk halus dengan amplitudo tetap antara -1 dan 1. Rentang waktu dibuat dengan np.linspace(0,1,1000), menghasilkan 1000 titik dari 0 hingga 1 detik. Selanjutnya nilai sinus dihitung dengan rumus np.pi(2*np.pi*f*t) di mana f adalah frekuensi awal (5 Hz), sehingga dalam rentang 1 detik terdapat 5 gelombang penuh. Kemudian grafik ditampilkan menggunakan plt.plot(t,x_continuous). Dengan memodifikasi frekuensi menjadi 7.5 Hz, jumlah gelombang penuh dalam rentang waktu 1 detik bertambah menjadi 7,5, membuat gelombangnya lebih rapat. Ketika frekuensi dinaikkan lagi menjadi 20 Hz, terdapat 20 gelombang penuh dalam 1 detik, menyebabkan bentuk gelombang terlihat semakin padat. Hal ini menunjukkan bahwa semakin tinggi frekuensi, semakin rapat gelombangnya, karena lebih banyak siklus terjadi dalam waktu yang sama. Sebaliknya, jika frekuensi diturunkan, gelombang menjadi lebih renggang.

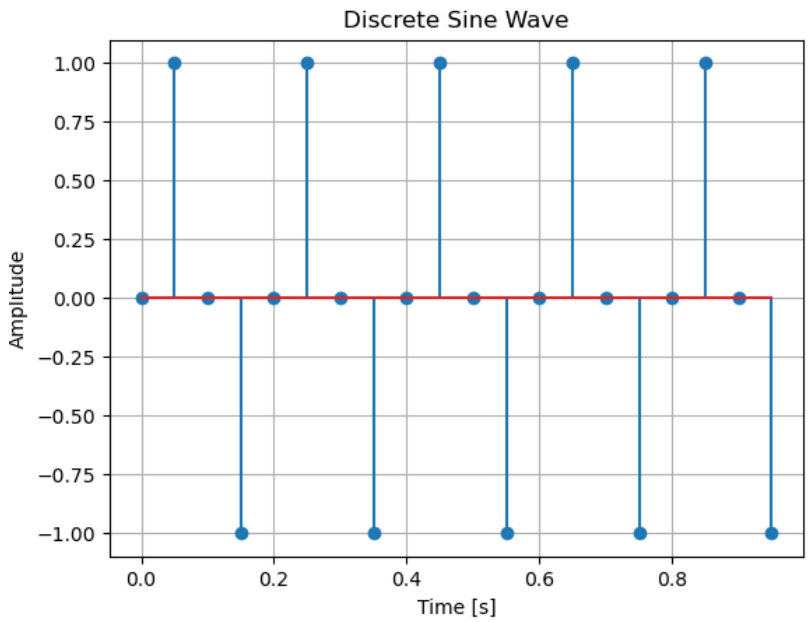
b. Plotting a Discrete-Time Sine Wave

- f = 5 Hz , fs = 20 Hz

```
import numpy as np
import matplotlib.pyplot as plt

f = 5
fs = 20
n = np.arange(0, 1, 1/fs)
x_discrete = np.sin(2 * np.pi * f * n)

plt.figure()
plt.stem(n, x_discrete)
plt.title('Discrete Sine Wave')
plt.xlabel('Time [s]')
plt.ylabel('Amplitude')
plt.grid(True)
plt.show()
```

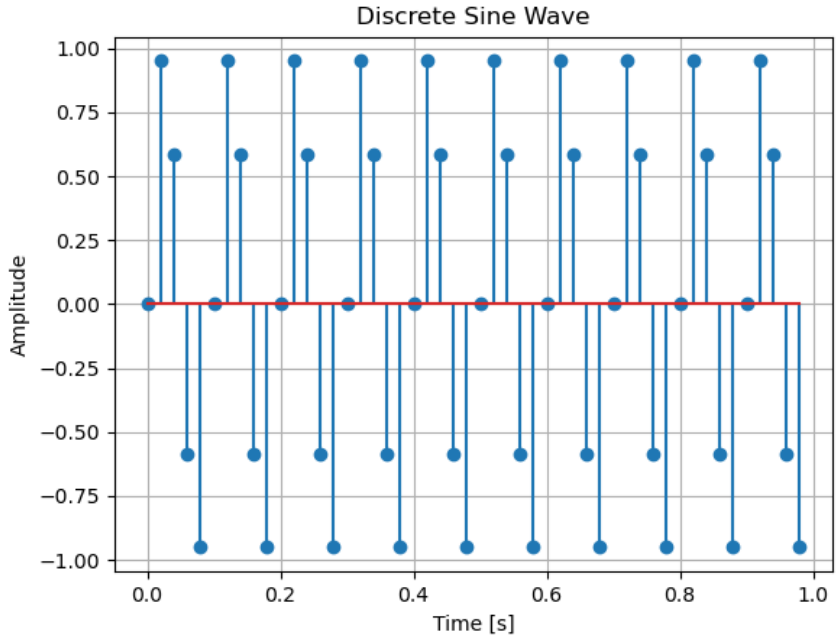


- f = 10 Hz, fs = 50 Hz

```
import numpy as np
import matplotlib.pyplot as plt

f = 10
fs = 50
n = np.arange(0, 1, 1/fs)
x_discrete = np.sin(2 * np.pi * f * n)

plt.figure()
plt.stem(n, x_discrete)
plt.title('Discrete Sine Wave')
plt.xlabel('Time [s]')
plt.ylabel('Amplitude')
plt.grid(True)
plt.show()
```

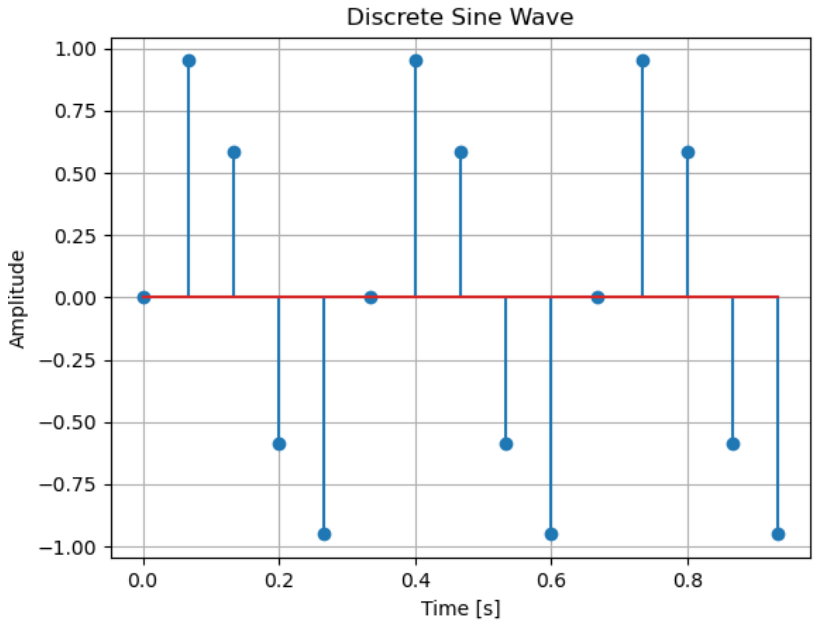


- f = 3 Hz, fs = 15 Hz

```
import numpy as np
import matplotlib.pyplot as plt

f = 3
fs = 15
n = np.arange(0, 1, 1/fs)
x_discrete = np.sin(2 * np.pi * f * n)

plt.figure()
plt.stem(n, x_discrete)
plt.title('Discrete Sine Wave')
plt.xlabel('Time [s]')
plt.ylabel('Amplitude')
plt.grid(True)
plt.show()
```



Analisa:

Di dalam percobaan ini, dilakukan visualisasi gelobang sinus diskret dengan mengambil sampel dari gelombang sinus kontinu. Frekuensi sampling yang cukup tinggi dibandingkan dengan frekuensi sinyal, akan membentuk gelombang yang masih terlihat. Samplingnya ditentukan dengan fs=20, maka gelombang diambil sampelnya setiap 1/20 detik. Rentang waktu dibuat dengan np.arange(0,1,1/fs),

menghasilkan titik-titik diskrit dengan interval 1/fs. Selanjutnya dihitung dengan np.sin(2*np.pi*f*n), menghasilkan beberapa titik dalam satu periode. Kemudian grafik ditampilkan dengan plt.stem(n,x_discrete) yang menggambarkan titik-titik sebagai garis vertical. Dengan memodifikasi nilai frekuensi sampling, seperti menggunakan fs = 20 Hz untuk f = 5 Hz, fs = 50 Hz untuk f = 10 Hz, dan fs = 15 Hz untuk f = 3 Hz, semakin tinggi frekuensi sampling, semakin banyak titik sampel yang diambil, sehingga bentuk sinyal diskrit lebih menyerupai sinyal kontinu. Semakin tinggi fs dibandingkan f, semakin baik sinyal diskrit merepresentasikan sinyal kontinu.

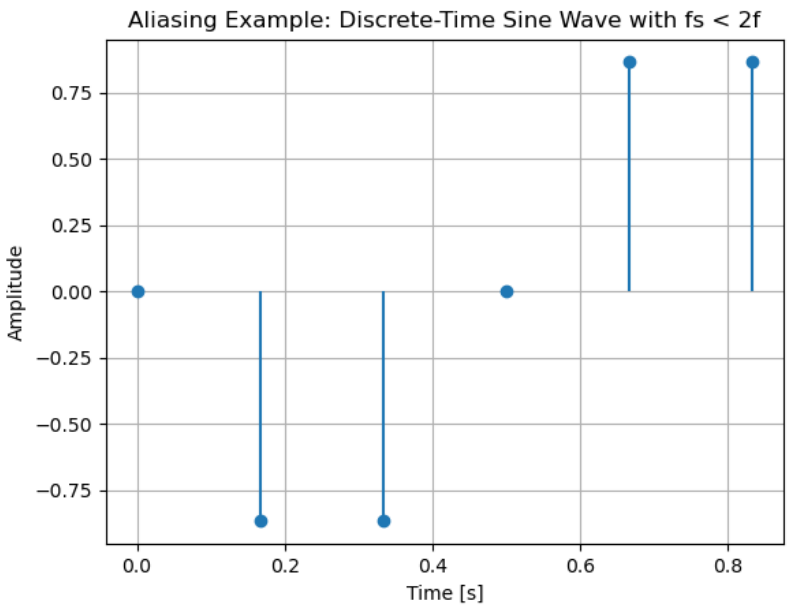
c. From Continuous to Discrete: The Sampling Process

- Aliasing, fs = 6 Hz

```
import numpy as np
import matplotlib.pyplot as plt

f = 5
fs_aliasing = 6
n_aliasing = np.arange(0, 1, 1/fs_aliasing)
x_aliasing = np.sin(2 * np.pi * f * n_aliasing)

plt.figure()
plt.stem(n_aliasing, x_aliasing, basefmt=" ")
plt.title("Aliasing Example: Discrete-Time Sine Wave with fs < 2f")
plt.xlabel("Time [s]")
plt.ylabel("Amplitude")
plt.grid(True)
plt.show()
```

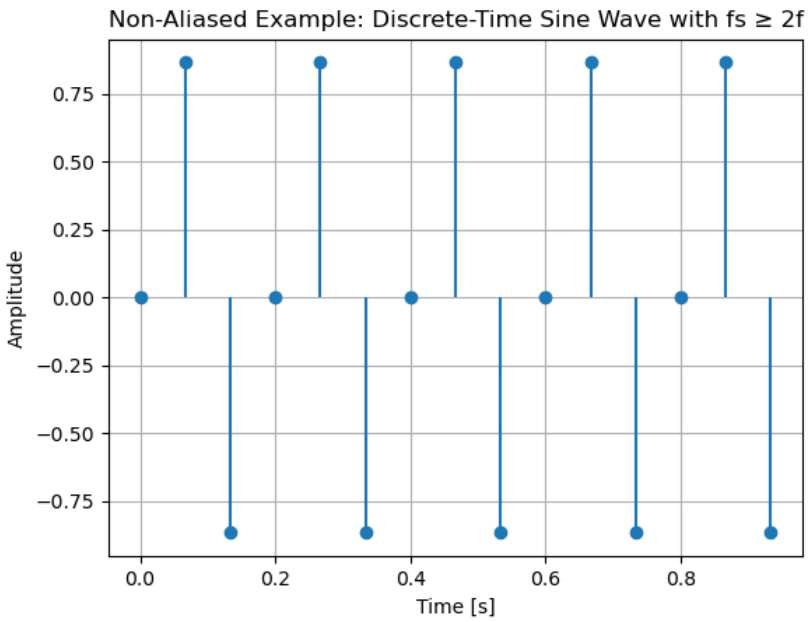


- Non-aliasing, fs = 15 Hz

```
import numpy as np
import matplotlib.pyplot as plt

f = 5
fs_noaliasing = 15
n_noaliasing = np.arange(0, 1, 1/fs_noaliasing)
x_noaliasing = np.sin(2 * np.pi * f * n_noaliasing)

plt.figure()
plt.stem(n_noaliasing, x_noaliasing, basefmt=" ")
plt.title("Non-Aliased Example: Discrete-Time Sine Wave with fs ≥ 2f")
plt.xlabel("Time [s]")
plt.ylabel("Amplitude")
plt.grid(True)
plt.show()
```



Analisa:

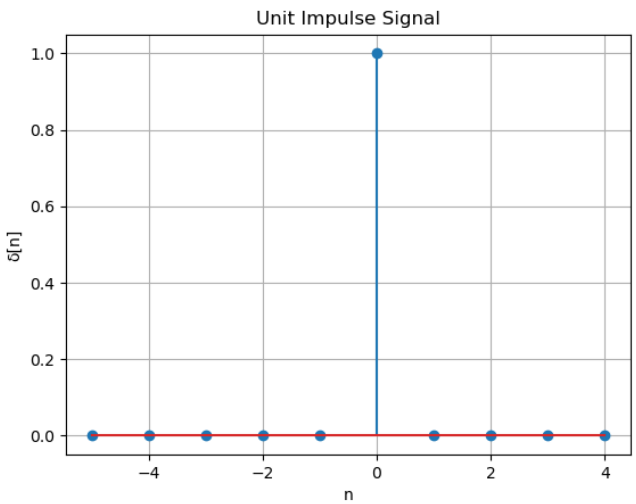
Di dalam percobaan ini, proses sampling ditunjukkan dengan mengambil sampel dari sinyal kontinu untuk membentuk sinyal diskret. Jika frekuensi sampling terlalu rendah, efek aliasing terjadi, bentuk sinyal yang dihasilkan tidak merepresentasikan sinyal aslinya dengan benar. Frekuensi sampling rendah seperti fs = 6 Hz untuk sinyal f = 5 Hz menyebabkan aliasing karena jumlah sampel yang diambil terlalu sedikit. Jika dibandingkan, dengan fs = 15 Hz, sinyal diskret lebih akurat karena jumlah sampel cukup banyak. Rentang waktu sampel dibuat menggunakan np.arange(0,1,1/fs), yang menghasilkan titik diskrit dengan interval 1/fs. Nilai sinus dihitung dengan np.sin(2*np.pi*f*n), kemudian divisualisasikan dengan plt.stem(n, x_discrete) untuk menampilkan titik-titik diskret. Ffek aliasing terlihat saat memilih fs yang terlalu rendah. Jika fs<2f (batas Nyquist), sinyal terlihat seperti memiliki frekuensi lebih rendah yang salah.

2. Unit impulse signal

```
import numpy as np
import matplotlib.pyplot as plt

n = np.arange(-5, 5, 1)
delta = np.zeros_like(n)
delta[5] = 1

plt.figure()
plt.stem(n, delta)
plt.title("Unit Impulse Signal")
plt.xlabel("n")
plt.ylabel("δ[n]")
plt.grid(True)
plt.show()
```

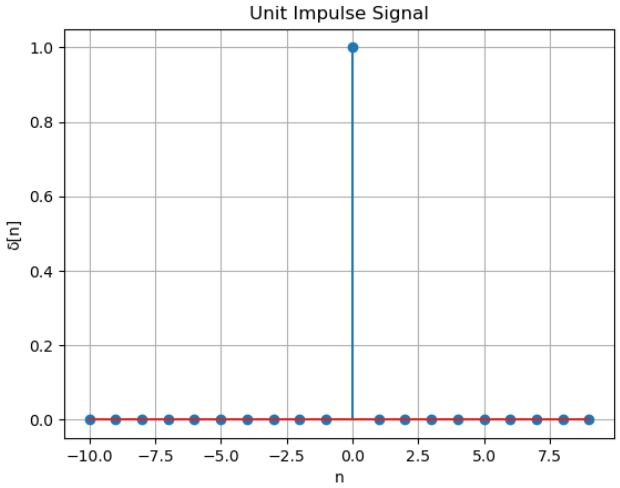


Modifikasi rentang n:

```
import numpy as np
import matplotlib.pyplot as plt

n = np.arange(-10, 10, 1)
delta = np.zeros_like(n)
delta[10] = 1

plt.figure()
plt.stem(n, delta)
plt.title("Unit Impulse Signal")
plt.xlabel("n")
plt.ylabel("δ[n]")
plt.grid(True)
plt.show()
```



Analisa:

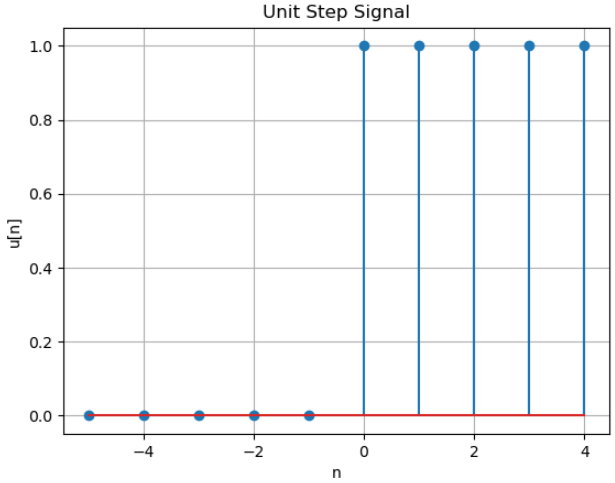
Di dalam percobaan ini, sinyal impulse unit dibuat di mana hanya satu titik yg memiliki nilai 1, sedangkan yang lainnya bernilai 0. Rentang waktu dibuat dengan n= np.arange(-5,5,1), dan array nol menggunakan delta=np.zeros_like(n). Nilai 1 ditempatkan pada indeks tengah dengan delta[5]=1, sehingga impuls muncul di n = 0. Dengan memodifikasi rentang n menjadi n=np.arange(-10,10,1), sinyal tetap memiliki satu titik dengan nilai 1, tetapi rentang total sinyal lebih luas. Perubahan ini tidak mengubah bentuk sinyal, hanya skala waktunya yang lebih panjang dan menjadi tidak sempurna.

3. Unit step signal

```
import numpy as np
import matplotlib.pyplot as plt

n = np.arange(-5, 5, 1)
step = np.heaviside(n, 1)

plt.stem(n, step)
plt.title("Unit Step Signal")
plt.xlabel("n")
plt.ylabel("u[n]")
plt.grid(True)
plt.show()
```

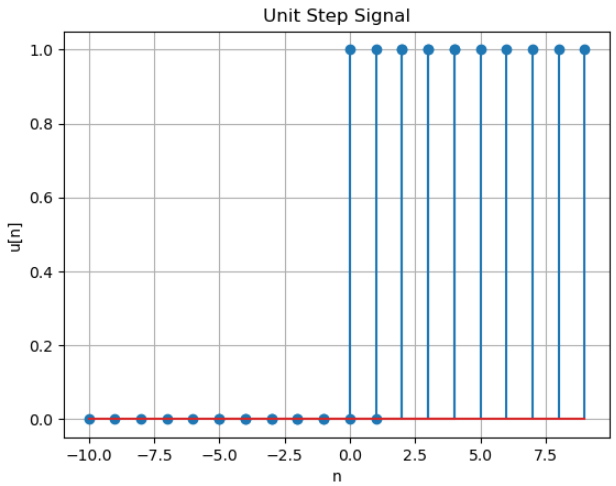


Modifikasi rentang n:

```
import numpy as np
import matplotlib.pyplot as plt

n = np.arange(-10, 10, 1)
step = np.heaviside(n - 2,1)

plt.stem(n, step)
plt.title("Unit Step Signal")
plt.xlabel("n")
plt.ylabel("u[n]")
plt.grid(True)
plt.show()
```



Analisa:

Di dalam percobaan ini, sinyal step unit digambarkan sebagai sinyal diskret yang berubah dari 0 menjadi 1 pada titik tertentu. Rentang waktu dibuat dengan n=np.arange(-5, 5, 1), dan sinyal step dihitung dengan np.heaviside(n,1), yang membuat perubahan terjadi di n = 0. Dengan memodifikasi rentang n menjadi n=np.arange(-10,10,1), cakupan sinyal lebih luas tanpa mengubah bentuknya.

Kemudian, dengan menggeser posisi perubahan nilai menggunakan np.heaviside(n-2,1), di n=2, menyebabkan sinyal bergeser ke kanan. Sehingga unit step dapat bergeser ke titik mana pun tergantung pada pergeserannya.

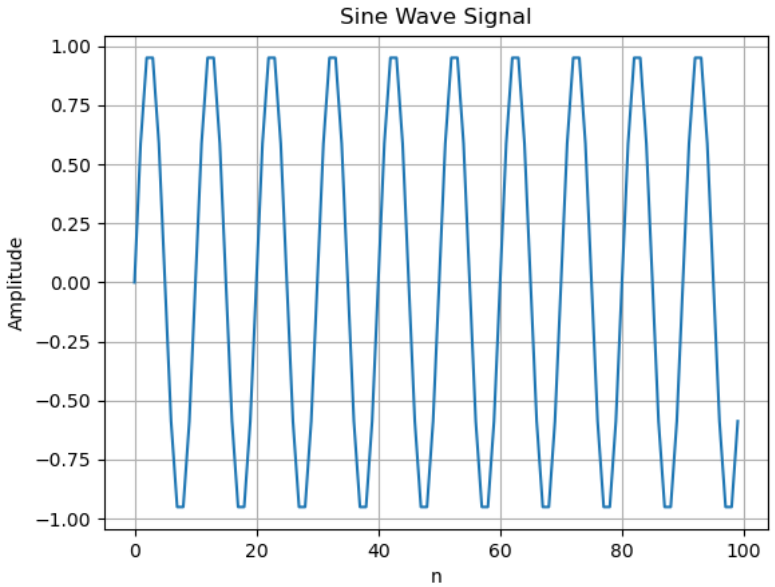
4. Sine and complex exponential signals

```
import numpy as np
import matplotlib.pyplot as plt

A = 1
f = 0.1
phi = 0
n = np.arange(0, 100, 1)

sine_wave = A * np.sin(2 * np.pi * f * n + phi)

plt.plot(n, sine_wave)
plt.title("Sine Wave Signal")
plt.xlabel("n")
plt.ylabel("Amplitude")
plt.grid(True)
plt.show()
```



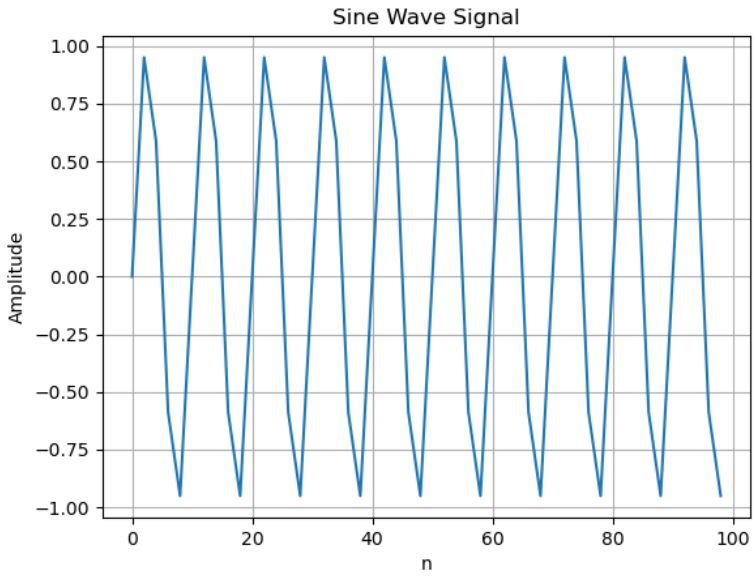
Modifikasi interval:

```
import numpy as np
import matplotlib.pyplot as plt

A = 1
f = 0.1
phi = 0
n = np.arange(0, 100, 2)

sine_wave = A * np.sin(2 * np.pi * f * n + phi)

plt.plot(n, sine_wave)
plt.title("Sine Wave Signal")
plt.xlabel("n")
plt.ylabel("Amplitude")
plt.grid(True)
plt.show()
```



Analisa:

Di dalam percobaan ini, gelombang sinus dan eksponensial kompleks dibuat menggunakan array diskret yang terdapaat osilasi. Rentang waktu dibuat dengan np.arange(0,100,1), yang menghasilkan titik sampel setiap 1 unit waktu. Selanjutnya sinus dihitung np.sin(2*np.pi*f*n), dan grafiknya ditampilkan menggunakan plt.plot(n, x). Dengan memodifikasi interval menjadi np.arange(0,100,2), jumlah titik sampel berkurang, menyebabkan gelombang terlihat lebih kasar dan tidak sempurna. Interval yang lebih besar berarti datanya lebih sedikit, sehingga sinyal menjadi lebih terpisah dan bentuknya tidak sehalus sebelumnya.

5. Discrete-Time Convolution

```
import torch

x = torch.tensor([1, 2, 3], dtype=torch.float32)
h = torch.tensor([0, 1, 0.5], dtype=torch.float32)

x = x.view(1, 1, -1)
h = h.view(1, 1, -1)

y = torch.nn.functional.conv1d(x, h, padding=2)
print(y.view(-1))
```

```
tensor([0.5000, 2.0000, 3.5000, 3.0000, 0.0000])
```

Modifikasi kernel:

```
import torch

x = torch.tensor([1, 2, 3], dtype=torch.float32)
h = torch.tensor([2, 6, 3], dtype=torch.float32)

x = x.view(1, 1, -1)
h = h.view(1, 1, -1)

y = torch.nn.functional.conv1d(x, h, padding=2)
print(y.view(-1))
```

```
tensor([ 3., 12., 23., 22.,  6.])
```

Analisa:

Di dalam percobaan ini, konvolusi waktu diskret diuji dengan sinyal input x = [1,2,3] dan kernel h = [0,1,0.5]. Konvolusi dihitung dengan np.convolve(x, h, mode='full'), menghasilkan tensor [0.5,2,3.5,3,0]. Sehingga setiap titik dalam sinyal input dikalikan dengan kernel dan dijumlahkan untuk mendapatkan nilai konvolusi di setiap titik waktu.

Ketika kernel diubah, h = [2,6,3], hasil konvolusi berubah menjadi [3,12,23, 22,6]. Perubahan terjadi karena bobot kernel lebih besar membuat nilai-nilai dalam sinyal input menjadi lebih signifikan. Semakin besar nilai kernel, semakin besar pengaruhnya terhadap hasil konvolusi.

6. Correlation

- Cross-Correlation

```
import torch

x = torch.tensor([1, 2, 3], dtype=torch.float32)
y = torch.tensor([4, 5, 6], dtype=torch.float32)

cross_corr = torch.nn.functional.conv1d(x.view(1, 1, -1), y.flip(0).view(1, 1, -1))
print(cross_corr.view(-1))
```

```
tensor([28.])
```

Modifikasi kernel:

```
import torch

x = torch.tensor([1, 2, 3], dtype=torch.float32)
y = torch.tensor([6, 7, 8], dtype=torch.float32)

cross_corr = torch.nn.functional.conv1d(x.view(1, 1, -1), y.flip(0).view(1, 1, -1))
print(cross_corr.view(-1))
```

```
tensor([40.])
```

- Auto-Correlation

```
import torch

x = torch.tensor([1, 2, 3], dtype=torch.float32)

auto_corr = torch.nn.functional.conv1d(x.view(1, 1, -1), x.flip(0).view(1, 1, -1))
print(auto_corr.view(-1))
```

```
tensor([10.])
```

Analisa:

Di dalam percobaan ini, korelasi digunakan untuk membandingkan kesamaan dari dua sinyal. Korelasi silang dilakukan dengan membandingkan sinyal input x = [1,2,3] dengan kernel y = [4,5,6] menggunakan np.correlate(x,y,mode='full'), yang menghasilkan tensor [28], menunjukkan terdapat kecocokan di antara kedua sinyal tersebut. Ketika diubah menjadi y = [6,7,8], hasil korelasi meningkat menjadi [40]. Sehingga sinyal yang lebih besar atau lebih mirip dengan sinyal input akan menghasilkan nilai korelasi yang lebih tinggi. Kemudian dilanjut pada auto-korelasi, sinyal x dikalikan dengan sinyal terbaliknya sendiri menggunakan np.correlate(x,x, mode='full'), yang menghasilkan tensor [10]. Maka semakin mirip suatu sinyal dengan versinya sendiri yang bergeser, semakin tinggi nilai auto-korelasinya.

III. KESIMPULAN

Pemrosesan Sinyal Digital (DSP) punya peran besar dalam menganalisis dan mengolah sinyal. Saat menggambarkan sinyal dalam bentuk kontinu atau diskrit, kita bisa melihat bagaimana frekuensi dan tingkat sampling memengaruhi bentuk gelombang yang dihasilkan. Jika samplingnya kurang tepat, bisa muncul aliasing yang membuat sinyal jadi tidak akurat. Sinyal seperti unit impulse dan unit step juga punya karakteristik khas yang sering dimanfaatkan dalam berbagai aplikasi teknis. Sedangkan, konvolusi membantu mengubah sinyal dengan menerapkan filter tertentu, korelasi juga berguna untuk mengenali pola dalam sinyal, yang berpengaruh dalam bidang pemrosesan gambar dan komunikasi digital.